📋 Presentation Notes — DBMS PBL

Library Circulation & Digital Resource Management System

> *These are your reference notes. Read through them before the presentation so you can explain each topic confidently in your own words.*

🔴 PART 1: PROBLEM STATEMENT

The One-Line Summary

> *"Libraries still run on registers and spreadsheets. That causes lost books, wrong fines, unfair reservations, and zero tracking of digital content. We're building a proper database to fix all of that."*

What's Actually Wrong? (The 5 Core Problems)

1. Books Get Lost Because Tracking is Manual

- Librarians write entries in a physical register or an Excel sheet.
- When a book is moved, returned, or misplaced — **nobody updates the record in real time**.
- Result: You go to the shelf, the book isn't there, and nobody knows where it went.

2. Digital Resources Are Completely Ignored

- Modern libraries have **e-books, e-journals, audiobooks** — not just physical books.
- Old systems have **no way** to catalog, track access, or even store metadata for these.
- Result: The library owns digital content but can't tell you who's using it or how popular it is.

3. Fines Are Calculated by Hand = Mistakes & Arguments

- A librarian has to manually count overdue days, look up the fine rate, and calculate the amount.
- This is **error-prone** — sometimes a student gets overcharged, sometimes undercharged.
- Result: Disputes, wasted time, and inconsistent enforcement.

4. Reservations Are Chaotic

- If 5 people want the same popular book, there's no fair queuing system.
- Who gets notified first? Who's been waiting longest? — **Nobody knows**.
- Result: The person who happens to check in at the right time gets the book, not the person who reserved first.

5. Data is Duplicated Everywhere

- The same member's info might be in the issue register, the fine register, and the membership register — **three different places**.
- If their phone number changes, it gets updated in one place but not the others.
- Result: **Data redundancy** (wasted storage) and **data inconsistency** (conflicting info).

How Our Database Solves Each Problem

PROBLEM	OUR SOLUTION
Books get lost	Every physical copy has a unique `AccessionNumber` and a `Status` field (Available / Issued / Lost / Damaged). Real-time tracking.
No digital resource tracking	Dedicated `DigitalResource` and `DigitalAccessLog` tables to catalog and track every access.
Manual fine calculation	`Fine` table with `CHECK (Amount ⩾ 0)`, linked to `Issue` records. Can be automated with triggers.
Unfair reservations	`Reservation` table with `QueuePosition` field — strict FIFO (first come, first served).
Data duplication	**Normalized schema** (3NF) — each piece of info is stored in exactly one place. Foreign keys link everything.

💬 How to Explain This in the Presentation

> *"The core problem we identified is that traditional library systems rely on manual processes — physical registers, spreadsheets, and human memory. This leads to five key issues: unreliable book tracking, no management of digital resources, error-prone fine calculations, chaotic reservation queues, and rampant data duplication. Our database system addresses each of these by centralizing all data in a properly normalized relational schema with strict constraints."*

🟢 PART 2: ER DIAGRAM

What is an ER Diagram? (Quick Refresher)

- **ER = Entity-Relationship**
- It's a **visual blueprint** of your database before you write any SQL.
- Shows: **Entities** (things you store data about), **Attributes** (properties of those things), **Relationships** (how things connect).

Our Entities — What Are They and Why Do We Need Them?

The Core Chain: Book → Edition → Copy

Think of it like this:

```
"Harry Potter" (Book)
   └── 1st Edition, Bloomsbury (Edition)
   |       └── Copy #001 on Shelf A3 (Copy)
   |       └── Copy #002 on Shelf A3 (Copy)
   └── 2nd Edition, Scholastic (Edition)
           └── Copy #003 on Shelf B1 (Copy)
```

- **Book** = The *abstract concept* of a book. Stores title, ISBN, language, etc.
- **Edition** = A *specific version* of that book (1st edition by Publisher X, 2nd edition by Publisher Y).
- **Copy** = A *physical item* sitting on a shelf. This is what a student actually borrows.

> **Why three tables instead of one?** Because one book can have multiple editions, and each edition can have multiple copies. If you cram all this into one table, you get massive data duplication.

The People: Member

- **Member** = Anyone registered at the library (Student, Faculty, External).
- Tracks their name, email, membership type, start/end dates, and how many books they're allowed to borrow simultaneously (`MaxBooksAllowed`).

The Actions: Issue, Reservation, Fine

- **Issue** = "Member X borrowed Copy Y on this date, due on that date." This is the core *circulation transaction*.
- **Reservation** = "Member X wants Book Y, but all copies are out. They're #3 in the queue."
- **Fine** = "Member X returned an issue late, so they owe ₹50." Linked to the specific Issue that caused it.

The Support Cast: Author, Publisher, Category

- **Author** = Stores author info. Connected to Books via a **junction table** (BookAuthor) because one book can have many authors and one author can write many books (**M:N**).
- **Publisher** = Connected to Edition (not Book!) because different editions can have different publishers.
- **Category** = Fiction, Science, History, etc. Also M:N with Book (a book can belong to multiple categories).

The Digital Side: DigitalResource, DigitalAccessLog

- **DigitalResource** = An e-book, audiobook, or e-journal version of a Book. Stores URL, file format, file size.
- **DigitalAccessLog** = Every time a member views or downloads a digital resource, it's logged here with timestamp and IP address.

Relationships — How Everything Connects

Key Relationships to Explain

RELATIONSHIP	TYPE	PLAIN ENGLISH
Book → Author	**Many-to-Many**	A book can have multiple authors. An author can write multiple books. Resolved via `BookAuthor` junction table.
Book → Category	**Many-to-Many**	A book can be in multiple categories. A category has multiple books. Resolved via `BookCategory` junction table.
Book → Edition	**One-to-Many**	One book can have many editions (1st, 2nd, 3rd...).
Edition → Publisher	**Many-to-One**	Many editions can come from the same publisher.
Edition → Copy	**One-to-Many**	One edition can have multiple physical copies in the library.
Copy → Issue	**One-to-Many**	A copy can be issued many times (over its lifetime), but only **one active issue at a time**.
Member → Issue	**One-to-Many**	A member can borrow many copies over time.
Member → Reservation	**One-to-Many**	A member can reserve multiple books.
Book → Reservation	**One-to-Many**	A popular book can have multiple people waiting for it.
Issue → Fine	**One-to-Many**	One issue can generate multiple fines (e.g., overdue + damaged).
Book → DigitalResource	**One-to-Many**	One book can have an e-book version, an audiobook, etc.
DigitalResource → DigitalAccessLog	**One-to-Many**	One resource can be accessed hundreds of times.

What's a Junction Table?

- When two entities have a **Many-to-Many** relationship, you can't directly represent it in a relational database.
- You create a **junction table** (also called bridge/associative table) that sits between them.
- Example: `BookAuthor` has two columns — `BookID` (FK → Book) and `AuthorID` (FK → Author). Together they form a **composite primary key**.

Participation Constraints (If Asked)

Total Participation (MUST exist):
- Every Copy MUST belong to an Edition
- Every Issue MUST link to a Copy AND a Member
- Every Fine MUST link to an Issue AND a Member

Partial Participation (MAY exist):
- A Member MAY or MAY NOT have any Issues (they could just be registered)
- A Book MAY or MAY NOT have a DigitalResource (print-only books)
- A Copy MAY or MAY NOT be issued (could be sitting on the shelf)

💬 How to Explain the ER Diagram in the Presentation

> *"Our ER model has 14 entities. The core flow is: a Book has Editions, each Edition has physical Copies, and Copies are what get Issued to Members. Alongside this, we track Reservations at the Book level, Fines at the Issue level, and digital content through DigitalResource with access logging. We handle many-to-many relationships like Book-Author and Book-Category through junction tables."*

🔵 PART 3: RELATIONAL SCHEMA

What is a Relational Schema? (Quick Refresher)

- The ER Diagram is the **conceptual design** (what entities exist and how they relate).
- The Relational Schema is the **logical design** (how you'll actually create tables in a database).

- It specifies: **table names, column names, data types, primary keys, foreign keys, constraints**.

How to Read the Schema Notation

```
Book(BookID, ISBN, Title, Language, TotalPages, PublicationYear)
```

- **Bold** = Primary Key (e.g., **BookID**)
- *Italic* = Foreign Key (e.g., *BookID* in Edition table references Book)
- ***Bold Italic*** = Both PK and FK (e.g., in junction tables like BookAuthor)

Our 14 Tables — Quick Reference

Independent Tables (No Foreign Keys)

These tables don't depend on any other table:

TABLE	PRIMARY KEY	KEY COLUMNS	PURPOSE
Book	BookID	ISBN (UNIQUE)	Core catalog of all books
Author	AuthorID	FirstName, LastName	Author information
Publisher	PublisherID	Name (UNIQUE)	Publisher details
Category	CategoryID	Name (UNIQUE)	Book categories/genres
Member	MemberID	Email (UNIQUE)	Library members

Junction Tables (Composite Primary Keys)

These resolve M:N relationships:

TABLE	PRIMARY KEY	FOREIGN KEYS	RESOLVES
BookAuthor	(BookID, AuthorID)	BookID → Book, AuthorID → Author	Book ↔ Author M:N
BookCategory	(BookID, CategoryID)	BookID → Book, CategoryID → Category	Book ↔ Category M:N

Dependent Tables (Have Foreign Keys)

TABLE	PRIMARY KEY	FOREIGN KEYS	PURPOSE
Edition	EditionID	BookID → Book, PublisherID → Publisher	Specific version of a book
Copy	CopyID	EditionID → Edition	Physical item on the shelf
Issue	IssueID	CopyID → Copy, MemberID → Member	Borrowing transaction
Reservation	ReservationID	BookID → Book, MemberID → Member	Waiting list entry
Fine	FineID	IssueID → Issue, MemberID → Member	Penalty record
DigitalResource	ResourceID	BookID → Book	E-book/audiobook metadata
DigitalAccessLog	LogID	ResourceID → DigitalResource, MemberID → Member	Usage tracking

Key Constraints — What They Mean and Why They Matter

Primary Key (PK)

- **What**: Uniquely identifies every row in a table.
- **Rule**: Can NEVER be NULL, must be unique.
- **Example**: `BookID` in Book table — no two books share the same ID.

Foreign Key (FK)

- **What**: A column that references the primary key of another table.
- **Why**: Enforces that you can't create an Issue for a CopyID that doesn't exist.
- **Example**: `Issue.CopyID` → `Copy.CopyID`. You can't issue a copy that isn't in the Copy table.

UNIQUE Constraint

- **What**: No two rows can have the same value in this column.
- **Different from PK**: A table can have multiple UNIQUE columns, but only one PK.
- **Our examples**: `Book.ISBN`, `Copy.AccessionNumber`, `Member.Email`, `Publisher.Name`, `Category.Name`

CHECK Constraint

- **What**: A rule that the value must satisfy.
- **Our examples**:
- `Fine.Amount ⩾ 0` — fines can't be negative
- `Book.TotalPages > 0` — a book must have at least 1 page
- `Edition.EditionNumber > 0` — edition must be 1st or higher
- `Reservation.QueuePosition ⩾ 1` — queue position starts at 1

DEFAULT Values

- **What**: If you insert a row without specifying this column, it gets this value.
- **Our examples**:
- `Copy.Status = 'Available'` — new copies start as available
- `Issue.Status = 'Active'` — new issues start as active
- `Member.IsActive = TRUE` — new members are active by default

ON DELETE Rules (Referential Actions)

RULE	MEANING	EXAMPLE
CASCADE	If parent is deleted, delete the children too	Deleting a Book → automatically deletes its Editions, BookAuthor entries, etc.
RESTRICT	Block the delete if children exist	Can't delete a Copy if there are Issue records for it

Why different rules?
- Use CASCADE for things that make no sense without the parent (a BookAuthor entry has no meaning without the Book).
- Use RESTRICT when you want to protect history (you shouldn't delete a Copy if issues/fines are recorded against it).

Data Types Cheat Sheet

DATA TYPE	WHAT IT STORES	USED FOR
INT	Whole numbers	IDs, page counts, queue positions
VARCHAR(n)	Text up to n characters	Names, emails, ISBNs
TEXT	Long text, no limit	Author bios, descriptions
DATE	Just the date (2024-01-15)	Issue dates, due dates
DATETIME	Date + time (2024-01-15 14:30:00)	Access timestamps, reservation times
DECIMAL(10,2)	Precise numbers with 2 decimal places	Fine amounts (₹49.50)
BOOLEAN	TRUE or FALSE	IsActive flag
ENUM	One value from a predefined list	Status fields ('Active', 'Returned', 'Overdue')

💬 How to Explain the Schema in the Presentation

> *"Our relational schema has 14 tables. 5 are independent master tables — Book, Author, Publisher, Category, and Member. 2 are junction tables that resolve many-to-many relationships. The remaining 7 are transaction/dependent tables. Every table has a primary key for entity integrity, foreign keys for referential integrity, and domain constraints like CHECK, UNIQUE, and DEFAULT to enforce our business rules at the database level."*

🎯 ANTICIPATED QUESTIONS & ANSWERS

Q: "Why did you separate Book, Edition, and Copy into three tables?"

> **A:** Because of normalization. A single book like "Harry Potter" can have multiple editions (1st edition, special edition), and each edition can have many physical copies. If we stored everything in one table, we'd duplicate the book title and ISBN for every single copy — that's data redundancy. Separating them achieves 3NF.

Q: "Why is Reservation linked to Book and not Copy?"

> **A:** Because when a member reserves, they want *any* available copy of that book, not a specific physical copy. The system assigns a specific copy when one becomes available.

Q: "Why does Fine reference both IssueID and MemberID?"

> **A:** `IssueID` tells us *which borrowing transaction* caused the fine. `MemberID` is technically redundant (you could get it from Issue → Member), but it's kept for **query performance** — you can directly find all fines for a member without joining through the Issue table.

Q: "What's a composite primary key?"

> **A:** When a single column can't uniquely identify a row, you combine two or more columns. In `BookAuthor`, neither `BookID` alone nor `AuthorID` alone is unique (a book can have many authors, an author can write many books). But the *combination* `(BookID, AuthorID)` is always unique.

Q: "What normalization form is your schema in?"

> **A:** Third Normal Form (3NF). Every non-key attribute depends on the key, the whole key, and nothing but the key. We've eliminated partial dependencies (via 2NF) and transitive dependencies (via 3NF).

📊 QUICK VISUAL SUMMARY

```
THE DATA FLOW:

  Book ──→ Edition ──→ Copy ──→ Issue ──→ Fine
   │                              │          │
   │                              └── Member ─┘
   │                                   │
   ├── Author (M:N via BookAuthor)     ├── Reservation
   ├── Category (M:N via BookCategory) │
   └── DigitalResource                 └── DigitalAccessLog
            │
            └── DigitalAccessLog
```

Total: 14 entities, 18 relationships, 5 UNIQUE constraints, 5 CHECK constraints, 15 foreign key links.

> *[!TIP]*
> ***Presentation tip:*** *Start with the Problem Statement to hook your audience, then show the ER Diagram to explain your design visually, and finally walk through the Schema to show how it all translates into actual database tables. The flow is:* ***Why*** *→* ***What*** *→* ***How****.*